

LAB2: 순차논리회로 실습

실험 전 보고서

학과: 전자전기컴퓨터공학부

이름/학번: 정현재/2023440127

작성일: 2026/09/21

대상: 카운터/클록 분주기/레지스터/시프트 레지스터/PISO/무어 머신/밀리 머신/세그먼트 스캔

검증 환경: VS Code VaporView

보고서 및 실험의 범위

업/다운 카운터, 클록 분주기, 레지스터, 시프트 레지스터, PISO, Moore 상태머신, Mealy 상태 머신 및 7세그먼트 자리 스캔의 총 8개 실험을 대상으로 Verilog HDL 설계와 사전 시뮬레이션을 정리했다. 실제 보드 구현·계측은 수업에서 수행할 항목으로 구분했다.

소스 식별

프로젝트: lab2

기준 커밋: 7468d23823f88e4fd143cf543e1050fca43259c

실행본: src/, sim/, constraints/

GitHub 주소: https://github.com/Jeong719/ECE2_Lab2.git

제출 태그: v1.0

1. 목적과 공통 검증 조건

실험 목적

카운터, 클록 분주기, 레지스터, 시프트 레지스터, PISO, 무어 머신, 밀리 머신, 세그먼트 자리 스캔 등 총 8종의 순차논리회로를 Verilog HDL로 설계하고, 시뮬레이션을 통해 진리표와 일치하는지 사전 검증한다.

관련 이론

순차회로의 동작은 회로의 입력, 출력, 현재 상태에 의해 결정된다. 출력과 다음상태는 입력과 현재 상태의 함수이다. [1] 이중 **무어 상태 머신**과 **밀리 상태 머신**의 차이점은 다음과 같다. 출력이 현재상태뿐만 아니라 입력에 의존하는 순차회로를 밀리 모델 회로라고 한다. 이와는 달리 출력이 현재상태에만 의존하면 1차원으로도 충분하다. 이러한 경우의 회로를 무어 모델 회로라고 한다. [2]

검증 환경과 실행 방법

사전 검증은 VS Code에서 Icarus Verilog를 이용하여 수행하고, 생성된 VCD 파일을 VaporView로 열어 파형을 분석한다.

통과 기준과 파형 분석 기준

정상 및 복구 실행은 테스트벤치의 모든 검사를 통과하여 LAB2_PASS가 출력되는 것을 통과 기준으로 한다. 실행 로그의 검사 수와 종료 시각을 기록하고 전체 파형과 각 실험마다 주요 기능이 담긴 구간의 확대 파형을 통해 조건에 따라 올바르게 출력됐는지 분석한다.

시간 단위와 검증 범위

파형의 시간 단위는 ns를 기준으로 해석하며, ps로 표시된 경우 1000ps를 1ns로 환산한다. TB의 10ns 클록은 기능 검증을 위한 조건이다.

2. 실험1: 업/다운 카운터

설계 파일: counter4.v, input_frontend.v, lab2_counter.v

설계와 예상 결과

본 회로는 4비트 상태값 value를 저장하는 업/다운 카운터다.

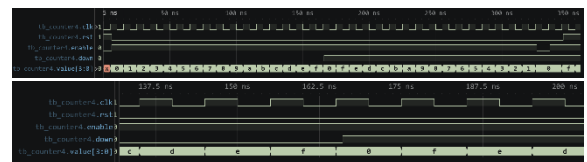
Rst=0, enable=1인 상태에서 클록 상승 에지가 발생하면, down=0 일 때 1 증가하고, down=1일 때 1 감소한다. Enable=0 이면 클록 상승 에지와 상관없이 이전 값을 유지하며, rst=1이면 다른 입력 조건과 관계 없이 다음 상승 에지에서 0으로 될 것으로 예상된다.

사전 검증과 파형 해석

```
VCD info: dumpfile wave.vcd opened for output.  
LAB2_PASS counter4 checks=36  
C:\Users\h3021\OneDrive\Documents\workspace\lab2_01_counter\src\lab2_counter4.v:22: $finish called at 356880 (ps)
```

사전 검증 결과 총 36회 검사한 것을 확인할 수 있다.

아래는 실험1의 파형과 주목해야 할 부분이다.



사진의 파형에서 주목해야 할 부분이 있다. Rst=0, enable=1, down=0 일 때 16진수로 1씩 증가하는 것을 확인할 수 있다. 이 때, 162.5ns와 175ns 사이 down=1이 된 이후 상승에지가 발생한 175ns에 증가하던 value 값이 다시 감소하는 방향으로 변화한 것을 확인할 수 있다.

위의 사진은 소스파일을 증가 연산과 관련된 코드인 12에서 2씩 증가하는 코드로 수정한 사진이다.

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 16ns에서 fail을 했다는 것이며, 이는 15ns에서 발생한 클록 상승 에지 이후 value 값이 2가 되어 tb파일의 기댓값 1과 충돌하여 실패한 것이다.

이후 다시 1씩 증가하는 코드로 수정한 후
성공적으로 pass가 뜨는 것을 확인했다.

Rst=1일 때 LED가 0을 표시하는지, 증가 및 감소 조작에 따라 값이 한 단계씩 변화하는지 확인한다. 또한, 최솟값 및 최댓값 도달 후 순환하는 구조가 정상적으로 작동되는지 확인한다.

설계 파일: clock_divider.v, input_frontend.v,
lab2_clock_divider.v

본 회로는 입력 클록을 분주하여 느린 주기의 출력 divided와 동작 허용 신호 tick을 생성한다. Count는 0부터 DIVISOR-1까지 반복하며, 분주 수 DIVISOR는 2 이상의

TB의 입력 주기가 10ns이고, DIVSOR=10이면 출력 주기는 100ns이며, 정상 반복 구간에서 HIGH와 LOW시간은 각각 50ns일 것으로 예상된다. Tick은 리셋이 해제되고 count=DIVISOR-1일 때 1이 될 것이며 다른 순차회로는 공통 클록을 유지하면서, tick이 1인 상승 에지에서만 상태를 갱신할 수 있을 것으로 예상된다.

사진의 파형에서는 tb에서 설정한대로 10분주가 되는 것을 확인할 수 있다. 55ns에서의 상승 에지와 함께 divided 값도 1이 된 것을 확인할 수 있다. 이후 divided 값이 다시 0이 될 때까지 clk의 상승에지가 총 5번 발생한 것을 확인할 수 있으며, 마지막 상승 에지에서는 tick의 값이 1이 된 것을 확인할 수 있다.

```

src > clock_divider.v
1 `timescale 1ns/1ps
2 // DIVISOR must be an even integer >= 2.
3 // divided is an observation output; use tick as an enable on clk.
4 module clock_divider #(parameter integer DIVISOR = 10)(
5     input wire clk, rst,
6     output reg divided,
7     output wire tick
8 );
9     localparam integer WIDTH = $clog2(DIVISOR);
10    reg [WIDTH-1:0] count;
11    assign tick = !rst && (count == DIVISOR - 1);
12    always @(posedge clk) begin
13        if (rst) begin
14            count <= 0;
15            divided <= 1'b0;
16        end else begin
17            if (count == DIVISOR - 1) count <= 0;
18            else count <= count + 1'b1;
19            if (count == DIVISOR/2) divided <= 1'b1;
20            else if (count == DIVISOR - 1) divided <= 1'b0;
21        end
22    end
23 endmodule

```

위의 사진은 소스 파일에서 divided가 1로 바뀌는 조건문에서 조건을 count가 divisor/2 - 1이 아닌 divisor/2로 수정한 사진이다.

```

LAB2_FAIL: Minimum divisor duty time=10000
FATAL: C:\Users\hjh\OneDrive\00000000000000000000\workspace\lab2_clock_divider\src\clock_divider.v:12: check fa
time: 10000 scope: tb_clock_divider.check

```

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 16ns에서 fail을 했다는 것이며, divided가 1이 되었어야 했을 시간에 0인 상태이기 때문에 실패한 것이다.

```

LAB2_PASS: clock_divider.check=120
C:\Users\hjh\OneDrive\00000000000000000000\workspace\lab2_clock_divider\src\clock_divider.v:12: #finish called at 430000

```

이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

LED의 변화 주기를 관찰하거나 영상으로 기록하여 예상 주기와 비교한다.

4. 실험3: 레지스터

설계 파일: register_pair.v, input_frontend.v, lab2_register.v

설계와 예상 결과

본 회로는 입력을 저장하는 stored와 저장값을 전달받는 value로 구성된다.

이를 수행하는 핵심 코드는 다음과 같다.

if (load) stored <= data_in;

if (transfer) value <= stored;

이 코드를 통해 상승 에지에서 load=1이면 data_in을 stored에 저장하고, transfer=1이면 stored를 value에 전달할 것으로 예상된다.

사전 검증과 파형 해석

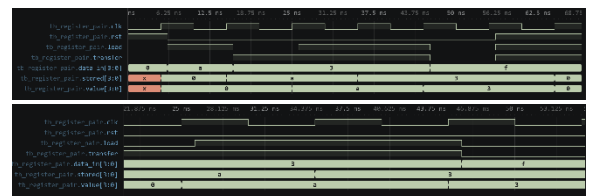
```

LAB2_PASS: register_pair.check=7
C:\Users\hjh\OneDrive\00000000000000000000\workspace\lab2_register\src\register_pair.v:19: #finish called at 60000 (ps)

```

사전 검증 결과 총 7회 검사 및 66ns가 소요된 것을 확인할 수 있다.

아래는 실험 3의 파형과 주목해야 할 부분에 대한 사진이다.



25ns 부근에서 load=0, transfer=1이다. 이 때, value의 값은 data_in이 아닌 이전에 저장된 stored 값으로 변하는 것을 확인할 수 있다.

35ns에서는 load=1, transfer=1이다. 이 때는 value의 값은 data_in의 값이 아닌 이전의 stored의 값으로 변하며, stored에 data_in의 값이 저장되는 것을 확인할 수 있다.

이후 45ns에서 value의 값이 stored에 저장된 3으로 변하는 것을 확인할 수 있다.

```

src > register_pair.v
1 `timescale 1ns/1ps
2 module register_pair(
3     input wire clk, rst, load, transfer,
4     input wire [3:0] data_in,
5     output reg [3:0] stored, value
6 );
7     always @(posedge clk) begin
8         if (rst) begin
9             stored <= 4'd0;
10            value <= 4'd0;
11        end else begin
12            if (load) stored <= data_in;
13            if (transfer) value <= data_in;
14        end
15    end
16 endmodule
17

```

위의 사진은 소스 파일의 13번째 줄에서

transfer=1일 때 value의 값을 stored 값이 아닌 data_in의 값으로 변하게끔 설정한 코드다.

```
LAB2_FAIL: transfer stored not live input time=20000
FATAL: C:\Users\h131\OneDrive\00000000000000000000\workspace\lab2_03_register\sim\tb_register_pair.v:12: check failed
Time: 20000 Scope: tb_register_pair.check
```

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 stored의 A 대신 data_in의 3을 value에 전달하였기 때문이다.

```
LAB2_PASS: register_pair.check
C:\Users\h131\OneDrive\00000000000000000000\workspace\lab2_03_register\sim\tb_register_pair.v:12: $finish called at 60000
```

이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

값 A를 저장한 뒤 입력 스위치를 3으로 바꾸고 transfer 조작을 수행하였을 때 출력이 현재 입력 3이 아닌 저장값 A가 되는지 확인한다.

5. 실험4: 시프트 레지스터

설계 파일: shift_register4.v,

input_frontend.v, lab2_shift_register.v

설계와 예상 결과

본 회로는 직렬 입력을 최상위 비트에 저장하고 기존 비트를 하위 비트 방향으로 이동시키는 4비트 시프트 레지스터다.

이 실험의 핵심 코드는 다음과 같다.

value <= {serial_in, value[3:1]};

이 코드를 통해 새 입력을 value[3]에 넣고 기존 value[3:1]을 다음 value [2:0]으로 이동시키며, 기존 최하위 비트는 제거될 것으로 예상된다.

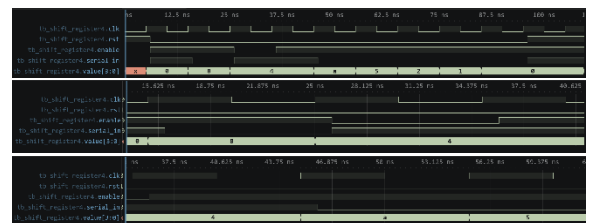
Enable=0이면 입력과 관계없이 저장값을 유지하며, rst=1이면 다음 상승 에지에서 0000으로 초기화될 것으로 예상된다.

사전 검증과 파형 해석

```
LAB2_PASS: shift_register.check
C:\Users\h131\OneDrive\00000000000000000000\workspace\lab2_05_shift_register\sim\tb_shift_register.v:12: $finish called at 100000 (ps)
```

사전 검증 결과 총 8회 검사 및 106ns가 소요된 것을 확인할 수 있다.

아래는 실험 4의 파형과 주목해야 할 부분에 대한 사진이다.



15ns에 enable=1 직렬 입력이 1인 상태에서 상승 에지가 발생한 이후 value의 값이 1000이 되는 것을 확인할 수 있다.

25ns에서 직렬 입력이 0인 상태에서 상승 에지가 발생한 이후 value의 값이 0100이 되는 것을 통해 기존의 비트가 하위 비트로 이동하는 것을 확인할 수 있다.

35ns에서 enable=0일 때는 직렬 입력이 1임에도 불구하고 기존의 0100을 유지하는 것을 확인할 수 있다.

```
src > @ shift_register4.v
1 timescale 1ns/1ps
2 // Serial input enters bit 3 and moves toward bit 0.
3 module shift_register4(
4 input wire clk, rst, enable, serial_in,
5 output reg [3:0] value
6 );
7 always @(posedge clk) begin
8 if (rst) value <= 4'd0;
9 else if (enable) value <= {value[2:0], serial_in};
10 end
11 endmodule
12
```

위의 사진은 소스 파일의 9번째 줄에서 직렬 입력의 값을 value의 최하위 비트에 저장되게끔 수정한 코드다.

```
LAB2_FAIL: input enters R08 time=10000
FATAL: C:\Users\h131\OneDrive\00000000000000000000\workspace\lab2_04_shift_register\sim\tb_shift_register4.v:12: check failed
Time: 10000 Scope: tb_shift_register4.check
```

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 원래 최상위 비트에 저장되었어야 할 직렬 입력값이 최하위 비트에 저장되어 tb의 기댓값과 달라 실패한 것이다.

```
LAB2_PASS: shift_register.check
C:\Users\h131\OneDrive\00000000000000000000\workspace\lab2_04_shift_register\sim\tb_shift_register4.v:12: $finish called at 200000 (ps)
```

이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

직렬 입력 스위치를 설정한 뒤 이동 조작을 통해 LED 비트가 이동하는 방향을 확인한다. 초기화 조작 시 모든 비트가 0으로 되는지, 최하위 비트에 1이 저장된 상태에서 다음 입력 시 1이 삭제되는지를 확인한다.

6. 실험5: PISO

설계 파일: piso4.v, input_frontend.v, lab2_piso.v

설계와 예상 결과

본 회로는 여러 비트를 병렬로 저장한 뒤 한 비트씩 직렬로 출력하는 회로다. Load=1일 때 4비트 입력을 저장하고, load가 0이면서 enable이 1일 때 저장값을 왼쪽으로 이동시키며 최하위 비트를 0으로 채운다.

핵심 코드는 다음과 같다.

assign serial_out = value[3];

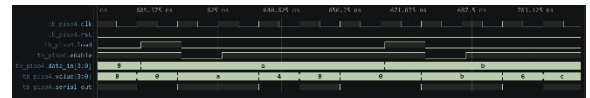
이 코드를 통해 출력이 value의 최상위 비트인 것을 확인할 수 있다.

사전 검증과 파형 해석

lab2_pass piso4 check=114
C:\Users\hhj3\OneDrive\00000000000000000000\workspace\lab2_05_piso\sim\tb_piso4.v:19: \$finish called at 976000 (1ps)

사전 검증 결과 총 114회 검사 및 976ns가 소요된 것을 확인할 수 있다.

아래는 실험 5의 파형과 주목해야 할 부분에 대한 사진이다.



615ns에서 load=1, enable=1, data_in=1010이다. 상승 에지 직후 value에 1010이 저장되며 출력이 1인 것을 통해 load가 우선하여 병렬 저장되는 것을 확인할 수 있다.

635ns에서 load=0, enable=1이기 때문에 상승 에지 직후 value가 한 번 시프트된 0100이며 최상위 비트 0이 출력되는 것을 확인할 수 있다.

645ns에서 동일한 상태기 때문에 한 번 더 시프트되어 1000으로 value가 변하며 최상위 비트 1이 출력되는 것을 확인할 수 있다.

```
src > @ piso4.v
1 timescale 1ns/1ps
2 module piso4(
3     input wire clk, rst, load, enable,
4     input wire [3:0] data_in,
5     output wire serial_out,
6     output reg [3:0] value
7 );
8     assign serial_out = value[0];
9     always @(posedge clk) begin
10         if (rst) value <= 4'd0;
11         else if (load) value <= data_in;
12         else if (enable) value <= {value[2:0], 1'b0};
13     end
14 endmodule
15
```

위의 사진은 소스 파일의 8번째 줄에서 value[3]을 value[0]으로 수정한 코드다.

LAB2_FAIL MSB first before edge time=86000
FATAL: C:\Users\hhj3\OneDrive\00000000000000000000\workspace\lab2_05_piso\sim\tb_piso4.v:12: check failed
Time: 86000 Scope: tb_piso4.check

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 출력을 value의 MSB 대신 LSB로 출력하게 되어 실패했기 때문이다.

lab2_pass piso4 check=114
C:\Users\hhj3\OneDrive\00000000000000000000\workspace\lab2_05_piso\sim\tb_piso4.v:19: \$finish called at 976000 (1ps)

이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

병렬 입력 1010을 설정하여 저장한 뒤 시

프트 전 직렬 출력 1을 확인하고 이후 시프트 조작마다 출력이 변하는 것을 확인한다.

7. 실험6: Moore 상태 머신

설계 파일: moore_cycle.v, input_frontend.v, lab2_moore.v

설계와 예상 결과

Moore 회로의 출력은 현재 상태에 의해 결정된다. 본 회로는 value가 상태이자 출력이다.

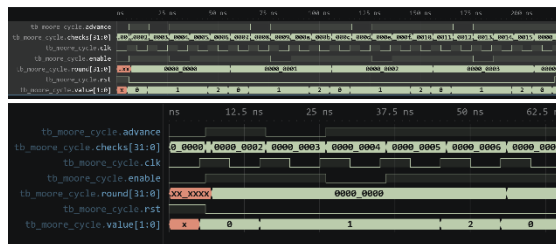
enable과 advance가 모두 1인 상승 에지에서 다음 상태로 전이할 것으로 예상된다.

사전 검증과 파형 해석

LAB2_PAS5 moore_cycle checks=23
C:\Users\hjh31\OneDrive\... \workspace\lab2_06_moore\sim\tb_moore_cycle.v:19: \$finish called at 226000 (1)

사전 검증 결과 총 23회 검사 및 226ns가 소요된 것을 확인할 수 있다.

아래는 실험 6의 파형과 주목해야 할 부분에 대한 사진이다.



15ns와 45ns에서 enable과 advance 모두 1일 때 value가 다음 상태로 전이되는 것을 확인할 수 있다.

25ns과 35ns에서는 advance와 enable이 각각 0이므로 value의 값이 다음 상태로 전이되지 않는 것을 확인할 수 있다.

```
moore_cycle.v X
src > moore_cycle.v
1 `timescale 1ns/1ps
2 module moore_cycle(
3     input wire clk, rst, enable, advance,
4     output reg [1:0] value
5 );
6     always @(posedge clk) begin
7         if (rst) value <= 2'b00;
8         else if (enable && advance) begin
9             case (value)
10                2'b00: value <= 2'b01;
11                2'b01: value <= 2'b00;
12                default: value <= 2'b00;
13            endcase
14        end
15    end
16 endmodule
```

위의 사진은 소스 파일의 11번째 줄에서 2'b10을 2'b00으로 바꾼 코드다.

LAB2_FAIL 51 to 52 time=46000
FATAL: C:\Users\hjh31\OneDrive\... \workspace\lab2_06_moore\sim\tb_moore_cycle.v:12: check failed
time: 46000 nscope: tb_moore_cycle.check

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 01인 상태 이후 10으로 전이되어야 하지만, 00으로 전이되었기 때문에 실패한 것이다.

LAB2_PAS5 moore_cycle checks=23
C:\Users\hjh31\OneDrive\... \workspace\lab2_06_moore\sim\tb_moore_cycle.v:19: \$finish called at 226000 (1ps)

이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

상태를 나타내는 LED가 유효한 조작마다 00->01->10->00으로 순환하는지 확인한다. 전이 조건을 만족하지 않았을 때는 상태가 바뀌지 않는 것을 확인한다.

8. 실험7: Mealy 상태 머신

설계 파일: mealy_toggle.v, input_frontend.v, lab2_mealy.v

설계와 예상 결과

Mealy 회로의 출력은 현재 상태와 현재 입력에 의해 결정되는 Moore에 비해서 복잡한 회로다.

본 회로는 enable=1, bit_in=1인 상승 에지에서 상태를 반전시킬 것으로 예상된다.
다음은 핵심코드다.

```
assign value = !bit_in ? 2'b00 : (state ? 2'b01 : 2'b10);
```

이는 출력 코드로 상승 에지가 없어도 입력이 바뀌면 출력이 바뀔 수 있다. 또한 enable이 0이어도 입력에 따른 출력 변화는 발생한다.

사전 검증과 파형 해석

LAB_PASS mealy_toggle checkall

사전 검증 결과 총 11회 검사 및 67ns가 소요된 것을 확인할 수 있다.

아래는 실험 7의 파형과 주목해야 할 부분에 대한 사진이다.



6ns 및 26ns에서 clk와 상관없이 bit_in, 즉 입력이 변하자 출력인 value가 변하는 것을 확인할 수 있다. 다만 state는 변하지 않는 것을 확인할 수 있다.

25ns에서 enable=1이 되자 state도 1로 변하며 이에 따라 value도 반전되는 것을 확인할 수 있다.

```
1 timescale 1ns/1ps
2 module mealy_toggle(
3     input wire clk, rst, enable, bit_in,
4     output reg state,
5     output wire [1:0] value
6 );
7 always @(posedge clk) begin
8     if (rst) state <= 1'b0;
9     else if (enable && bit_in) state <= ~state;
10 end
11 // Reset changes state at the rising edge; value is combinational
12 assign value = !bit_in ? 2'b00 : (state ? 2'b10 : 2'b01);
13 endmodule
14
```

위의 사진은 소스 파일의 12번째 줄에서 2'b10과 2'b01의 위치를 바꾼 코드다.

LAB2_FAIL 50 input changes between clocks time=7000
FAIL: C:\Users\hhj\OneDrive\... workspace\lab2_07_mealy\sim\b_mealy_toggle.vv:12: \$finish called at 67000 (1ps)
time: 7000 Scope: tb_mealy_toggle.check

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다. 이는 상태 S0에서 입력이 1일 때 예상 출력 10 대신 01이 나왔기 때문에 실패한 것이다.

LAB_PASS mealy_toggle checkall

이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

상태와 출력 LED를 구분하여 입력 스위치만 바꿨을 때 출력이 변하는지 확인하고 같은 입력에서도 상태가 바뀌면 출력도 변화하는지 확인한다.

9. 실험8: 7세그먼트 자리 스캔

설계 파일: segment_scan8.v,

input_frontend.v, lab2_segment_scan.v

설계와 예상 결과

본 회로는 32비트 digits를 4비트씩 나누어 8개 자리의 표시값으로 사용한다. Index는 현재 자리 번호이며, 다음 코드로 해당 자리의 숫자를 선택한다.

```
nibble = digits >> (index * 4);
```

자리 선택은 다음 코드로 이루어진다.

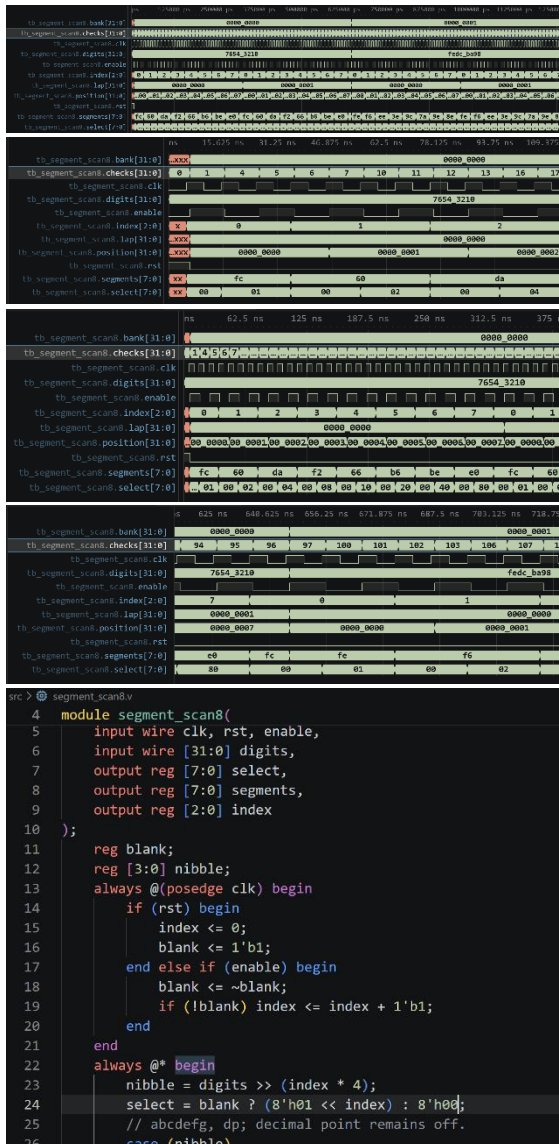
```
select = blank ? 8'h00 : (8'h01 << index);
```

사전 검증과 파형 해석

LAB_PASS segment_scan8 checkall

사전 검증 결과 총 194회 검사 및 1306ns가 소요된 것을 확인할 수 있다.

아래는 실험 8의 파형과 주목해야 할 부분에 대한 사진이다.



위의 사진은 소스 파일의 9번째 줄에서

```
lab2_1411 reset blank digit zero time=0000
FATAL: C:\Users\lhh3\OneDrive\0000\0000\00000000\workspace\lab2_08_segment_scan\src\th_segment_scan.v:12: check f
time: 6800 Scope: tb_segment_scan.check
```

해당 코드로 진행하면 터미널에 위와 같은 문구가 나타나는 것을 확인할 수 있다.

```
lab2_1411 segment_scan check=254
FATAL: C:\Users\lhh3\OneDrive\0000\0000\00000000\workspace\lab2_08_segment_scan\src\th_segment_scan.v:39: $finish called at 100000
```

이후 다시 코드를 수정한 후 성공적으로 pass가 뜨는 것을 확인했다.

보드에서 확인할 것

자리 순서, 숫자 패턴, 입력 변경에 따른 표시 변화 및 깜빡임, 잔상을 관찰한다.

10.실험 수행 계획

1단계(프로젝트 생성 및 코딩)

Vivado를 실행하여 실습 대상 회로의 RTL 프로젝트를 생성하고 Design Sources에 Verilog 모듈 코드 작성

2단계(테스트벤치 구동 및 기능 검증)

Simulation Sources에 테스트벤치를 추가하고 Run Behavioral Simulation을 실행하여 Vivado 내장 시뮬레이터로 출력파형 확인

3단계(결과 파형 분석 및 검증)

Vivado Waveform 창에서 입력 벡터에 따른 출력값의 변화를 확인하고, 이론적 진리표와 일치하는 대조하여 시뮬레이션 통과 확인

11.참고 문헌

[1] M. Morris Mano, Charles R. Kime, Tom Martin, 『디지털논리와 컴퓨터설계』, 제5판, 양세양·최명렬·최석림·하경재 역, 퍼스트북, 2016, pp. 198

[2] M. Morris Mano, Charles R. Kime, Tom Martin, 『디지털논리와 컴퓨터설계』, 제5판, 양세양·최명렬·최석림·하경재 역, 퍼스트북, 2016, pp. 200